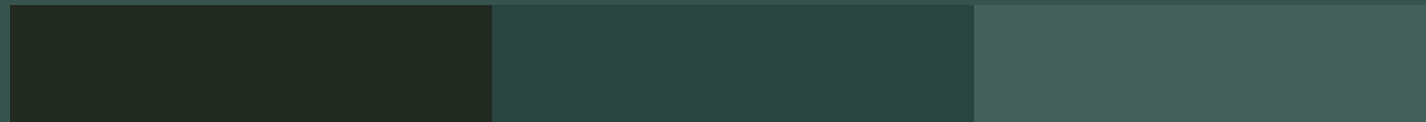


COMPILER

Araiza Valdés Diego Antonio
Arroyo Solano Victor Julian
Jaramillo Rodríguez Leslie Citlalli
Salas Hernández Camila Alexandra
Velazquez Caudillo Osbaldo



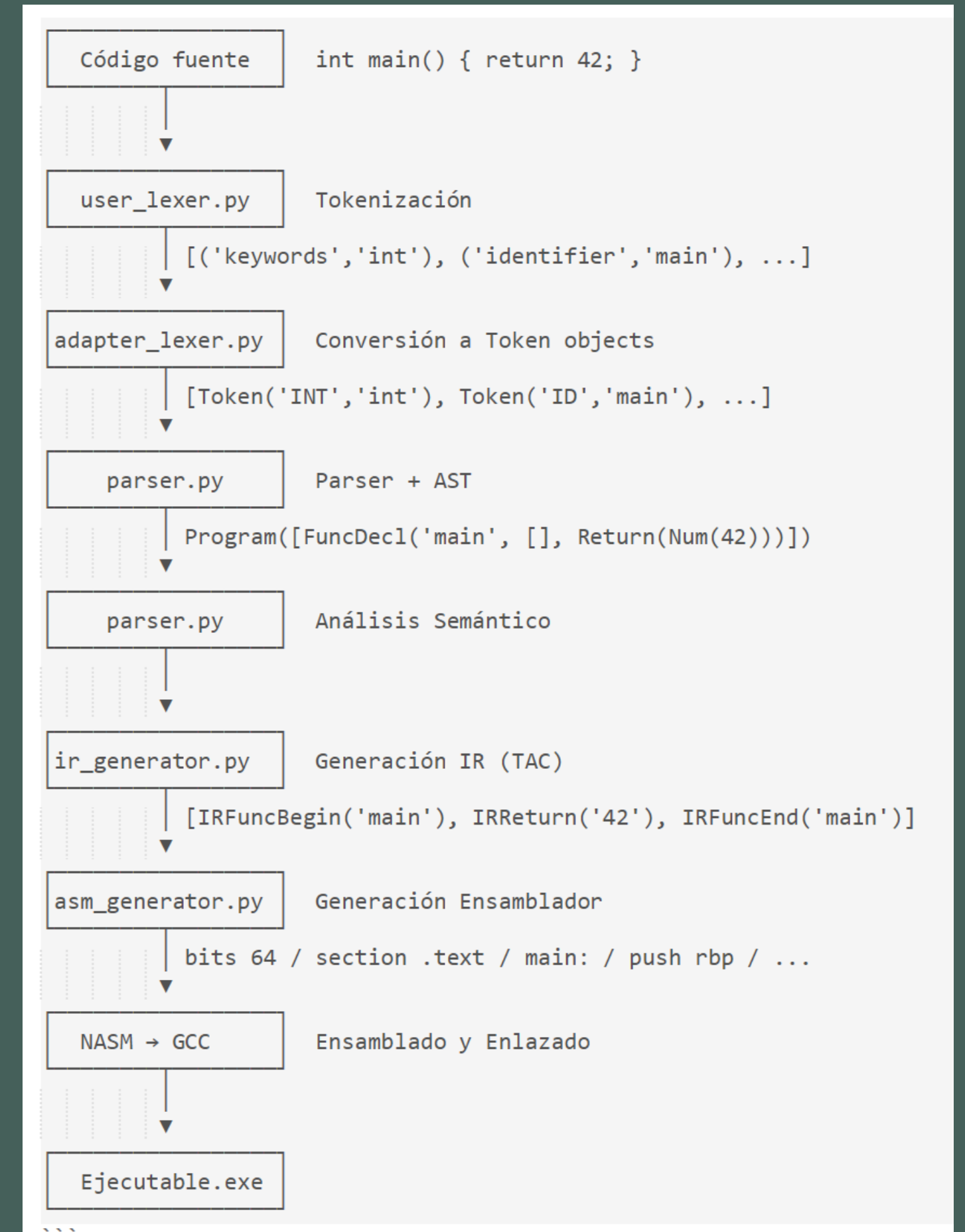
Objectives

- Generate Three-Address Code (TAC) from the AST.
- Translate TAC to x86-64 NASM assembly using Windows calling convention.
- Produce correct function prologues/epilogues and manage stack frames.
- Support variables, expressions, control flow, functions, and returns.
- Handle string literals and data section generation.
- Assemble with NASM (.obj) and link with GCC to create .exe files.
- Provide CLI options to view IR and assembly for debugging.
- Report clear errors at every compilation stage.



What our compiler does

- Translates C-like source code into Windows x86-64 executables.
- Implements all 6 major compiler phases
- Uses Three-Address Code (TAC) as Intermediate Representation
- Generates NASM x86-64 assembly
- Produces .exe files using NASM + GCC (MinGW-w64)
- Includes debugging tools for IR and ASM inspection



Supported language features

C-like syntax, including:

- Variable declarations (int, bool, float, string)
- Arithmetic + logical expressions
- Control flow: if / else, while, for
- Functions: declaration, parameters, return values
- Function calls with argument passing
- String literals
- Assignments, comparisons, boolean operations

Implementation

- `main.py` — Manages the entire pipeline
- `lexer/adaptlex.py` — Lexical analysis
- `parser.py` — Syntax + semantic analysis
- `ir_generator.py` — AST $\rightarrow$ TAC
- `asm_generator.py` — TAC $\rightarrow$ NASM x86-64

Error reporting

Error messages for:

- Lexical errors
- Syntax errors
- Semantic/type errors
- TAC generation issues
- Assembly generation errors
- Toolchain failures (NASM, GCC)

TEST

COMPILER

Test 1

```
int main() {  
    int a = 10;  
    int b = 5;  
    int suma = a + b;  
    int resta = a - b;  
    int mult = a * b;  
    int div = a / b;  
    int mod = a % b;  
  
    int resultado = (suma + resta) * mult - div + mod;  
    // resultado = (15 + 5) * 50 - 2 + 0 = 20 * 50 - 2 = 998  
  
    return resultado;  
}
```


Test 1

```
> py main.py tests\Test_E1.c --show-ir --show-asm ; Arquitectura: x86-64 (64 bits)
=== FASE 1: Análisis Léxico === ; Sintaxis: NASM (Intel)
✓ Tokens generados: 70 tokens

=== FASE 2: Análisis Sintáctico ===
✓ AST generado correctamente

=== FASE 3: Análisis Semántico ===
✓ Análisis semántico completado

=== FASE 4: Generación de Código Intermedio ===
✓ Código IR generado: 20 instrucciones

--- Código Intermedio (IR) ---
func main()
  a = 10
  b = 5
  t0 = a + b
  suma = t0
  t1 = a - b
  resta = t1
  t2 = a * b
  mult = t2
  t3 = a / b
  div = t3
  t4 = a % b
  mod = t4
  t5 = suma + resta
  t6 = t5 * mult
  t7 = t6 - div
  t8 = t7 + mod
  resultado = t8
  return resultado
endfunc main
--- Fin IR ---

=== FASE 5: Generación de Código Ensamblador ===
✓ Código ensamblador generado

--- Código Ensamblador (x86-64) ---
; Generado por el compilador

bits 64
default rel

section .text
  extern printf
  extern scanf
  extern exit

  global main

main:
  push rbp
  mov rbp, rsp
  sub rsp, 64 ; Espacio para variables locales

  mov QWORD [rbp-8], 10
  mov QWORD [rbp-16], 5
  mov rax, QWORD [rbp-8]
  mov rbx, QWORD [rbp-16]
  add rax, rbx
  mov QWORD [rbp-24], rax
  mov rax, QWORD [rbp-24]
  mov QWORD [rbp-32], rax
  mov rax, QWORD [rbp-8]
  mov rbx, QWORD [rbp-16]
  sub rax, rbx
  mov QWORD [rbp-40], rax
  mov rax, QWORD [rbp-40]
  mov QWORD [rbp-48], rax
  mov rax, QWORD [rbp-8]
  mov rbx, QWORD [rbp-16]
  imul rax, rbx
  mov QWORD [rbp-56], rax
  mov rax, QWORD [rbp-56]
  mov QWORD [rbp-64], rax
  mov rax, QWORD [rbp-8]
  mov rbx, QWORD [rbp-16]
  xor rdx, rdx
  idiv rbx
  mov QWORD [rbp-72], rax
  mov rax, QWORD [rbp-72]
  mov QWORD [rbp-80], rax
  mov rax, QWORD [rbp-8]
  mov rbx, QWORD [rbp-16]
  xor rdx, rdx
  idiv rbx
  mov rax, rdx
  mov QWORD [rbp-88], rax
  mov rax, QWORD [rbp-88]
  mov QWORD [rbp-96], rax
  mov rax, QWORD [rbp-32]
  mov rbx, QWORD [rbp-48]
  add rax, rbx
  mov QWORD [rbp-104], rax
  mov rax, QWORD [rbp-104]
  mov rbx, QWORD [rbp-64]
  imul rax, rbx
  mov QWORD [rbp-112], rax
  mov rax, QWORD [rbp-112]
  mov rbx, QWORD [rbp-80]
  sub rax, rbx
  mov QWORD [rbp-120], rax
  mov rax, QWORD [rbp-120]
  mov rbx, QWORD [rbp-96]
  add rax, rbx
  mov QWORD [rbp-128], rax
  mov rax, QWORD [rbp-128]
  mov QWORD [rbp-136], rax
  mov rax, QWORD [rbp-136]
  jmp .end_main
.end_main:
  mov rsp, rbp
  pop rbp
  ret

--- Fin ASM ---

✓ Archivo ensamblador guardado: Test_E1.asm

=== FASE 6: Ensamblado y Enlazado ===
Ensamblando con NASM...
✓ Objeto generado: Test_E1.obj
Enlazando con GCC...
✓ Ejecutable generado: Test_E1.exe

=====
✓ COMPILACIÓN COMPLETADA EXITOSAMENTE
=====

Ejecutable: Test_E1.exe
Para ejecutar: .\Test_E1.exe
```

Test 2

```
int main() {
    int x = 15;
    int y = 10;
    int resultado = 0;

    // Comparaciones relacionales
    if (x > y) {
        resultado = 1;
    } else {
        resultado = 0;
    }

    int mayor = 0;
    if (x >= 20) {
        mayor = 20;
    } else {
        mayor = x;
    }

    int igual = 0;
    if (x == y) {
        igual = 1;
    } else {
        igual = 0;
    }

    int diferente = 0;
    if (x != y) {
        diferente = 1;
    } else {
        diferente = 0;
    }

    // resultado final: 1 + 15 + 0 + 1 = 17
    return resultado + mayor + igual + diferente;
}
```

Test 2

```
> py main.py tests\Test_E2.c
=== FASE 1: Análisis Léxico ===
✓ Tokens generados: 122 tokens

=== FASE 2: Análisis Sintáctico ===
✓ AST generado correctamente

=== FASE 3: Análisis Semántico ===
✓ Análisis semántico completado

=== FASE 4: Generación de Código Intermedio ===
✓ Código IR generado: 44 instrucciones

=== FASE 5: Generación de Código Ensamblador ===
✓ Código ensamblador generado
✓ Archivo ensamblador guardado: Test_E2.asm

=== FASE 6: Ensamblado y Enlazado ===
Ensamblando con NASM...
✓ Objeto generado: Test_E2.obj
Enlazando con GCC...
✓ Ejecutable generado: Test_E2.exe

=====
✓ COMPILACIÓN COMPLETADA EXITOSAMENTE
=====

Ejecutable: Test_E2.exe
Para ejecutar: .\Test_E2.exe
```

Test 3

```
int main() {
    int suma = 0;
    int contador = 1;

    // While loop - suma de 1 a 5
    while (contador <= 5) {
        suma = suma + contador;
        contador = contador + 1;
    }
    // suma = 1+2+3+4+5 = 15

    // For loop - resta de 5 a 1
    int resta = 0;
    int i = 0;
    for (i = 5; i > 0; i = i - 1) {
        resta = resta + i;
    }
    // resta = 5+4+3+2+1 = 15

    // Operación unaria
    int negativo = -10;
    int positivo = -negativo; // positivo = 10

    // resultado: 15 + 15 + 10 = 40
    return suma + resta + positivo;
}
```

Test 3

```
> py main.py tests\Test_E3.c --asm-only
=== FASE 1: Análisis Léxico ===
✓ Tokens generados: 90 tokens

=== FASE 2: Análisis Sintáctico ===
✓ AST generado correctamente

=== FASE 3: Análisis Semántico ===
✓ Análisis semántico completado

=== FASE 4: Generación de Código Intermedio ===
✓ Código IR generado: 34 instrucciones

=== FASE 5: Generación de Código Ensamblador ===
✓ Código ensamblador generado
✓ Archivo ensamblador guardado: Test_E3.asm

✓ Compilación completada (solo ASM)
```

Test 4

```
int main() {  
    int x = 10;  
    int y = x @ 5; // @ no es un operador válido  
    return y;  
}
```

Test 4

```
> py main.py tests\Test_ER1.c  
=== FASE 1: Análisis Léxico ===  
X Error léxico: Carácter inesperado '@' en L10 C15 cerca de: '@ 5; // @ no es un '
```

Test 5

```
int main() {  
    int x = 10;  
    int y = 20;  
    int resultado = x + y + z; // 'z' no está declarada  
    return resultado;  
}
```

Test 5

```
> py main.py tests\Test_ER3.c  
=== FASE 1: Análisis Léxico ===  
✓ Tokens generados: 29 tokens  
  
=== FASE 2: Análisis Sintáctico ===  
X Error semántico: Uso de variable no declarada 'z'
```